

Documentație 5

Sistem de Achiziție, Procesare
și Afișare a Temperaturii
folosind Microcontrolerul
8051

Student: Șișca Silviu-Claudiu

Anul: III, 2133;

Cuprins

1. Cerința proiectului	3
2. Diagrama bloc de funcționare.....	4
3. Preluarea Datelor de la ADC.....	6
4. Interpretarea Datelor.....	7
5. Afișarea Datelor pe LCD.....	9
6. Simulări.....	10
7. Cod C & Assembly.....	15
8. Concluzii.....	23

1. Cerința Proiectului

Obiectivul acestui proiect este proiectarea și implementarea software a unui sistem cu microcontroler capabil să citească o temperatură, să o proceseze și să o afișeze cu o precizie de o zecimală.

Microcontrolerul trebuie programat în C, respectiv Assembly, astfel încât să îndeplinească următoarele funcționalități:

- Preluarea datelor: trebuie să comunice cu convertorul analog-digital (ADC) pentru a prelua datele brute
- Interpretarea datelor: valoarea digitală brută trebuie prelucrată matematic de către microcontroler pentru a fi convertită în grade Celsius, scalate corect pentru intervalul dorit de temperatură
- Afișarea datelor: rezultatul obținut în urma interpretării trebuie formatat și trimis către modulul de afișare (LCD)

2. Diagrama bloc de funcționare

Diagrama bloc de funcționare a software-ului dezvoltat pentru microcontroler este prezentată în Figura 1. Aceasta prezintă procesul de execuție a programului de la pornire până la bucla infinită de monitorizare a temperaturii. Execuția este structurată în 2 etape principale: etapa de inițializare și bucla principală de funcționare.

După pornirea sistemului, programul începe cu inițializarea porturilor microcontrolerului. Urmează inițializarea modulului LCD, proces în care sunt trimise secvențial comenzile de configurare. Pentru eficientizarea execuției, textul “Temperature:” este afișat o singură dată, înainte de a intra în bucla principală.

Sistemul apoi intră într-un ciclu infinit de achiziție, procesare și afișare.

Se plasează cursorul LCD-ului pe poziția dorită, adică imediat după textul static, astfel la momentul afișării se va suprascrie valoarea veche cu cea nouă.

Microcontrolerul trimite un impuls de comandă convertorului analog-digital pentru a iniția un nou proces de conversie.

Blocul de decizie “Conversie finalizată?” reprezintă soluția pentru sincronizarea microcontrolerului și a ADC-ului. Sistemul implementează o tehnică de polling hardware, monitorizând constant starea pinului de întrerupere. Cât timp conversia este în curs, programul rămâne blocat într-o buclă de așteptare (ramura NU). Când ADC-ul termină conversia va semnaliza acest aspect microcontrolerului, iar execuția continuă (ramura DA).

Odată finalizată conversia, valoarea brută pe 8 biți este citită de pe magistrala de date și trimisă către blocul de “Procesare Valoare”. Aici se aplică un algoritm matematic pentru convertirea valorii brute în grade Celsius și se extrag cifrele individuale (sute, zeci, unități). În final, cifrele procesate sunt convertite în format ASCII și afișate pe ecran, inserându-se și punctul zecimal. Ciclul se încheie cu un “Delay” de câteva sute de milisecunde.

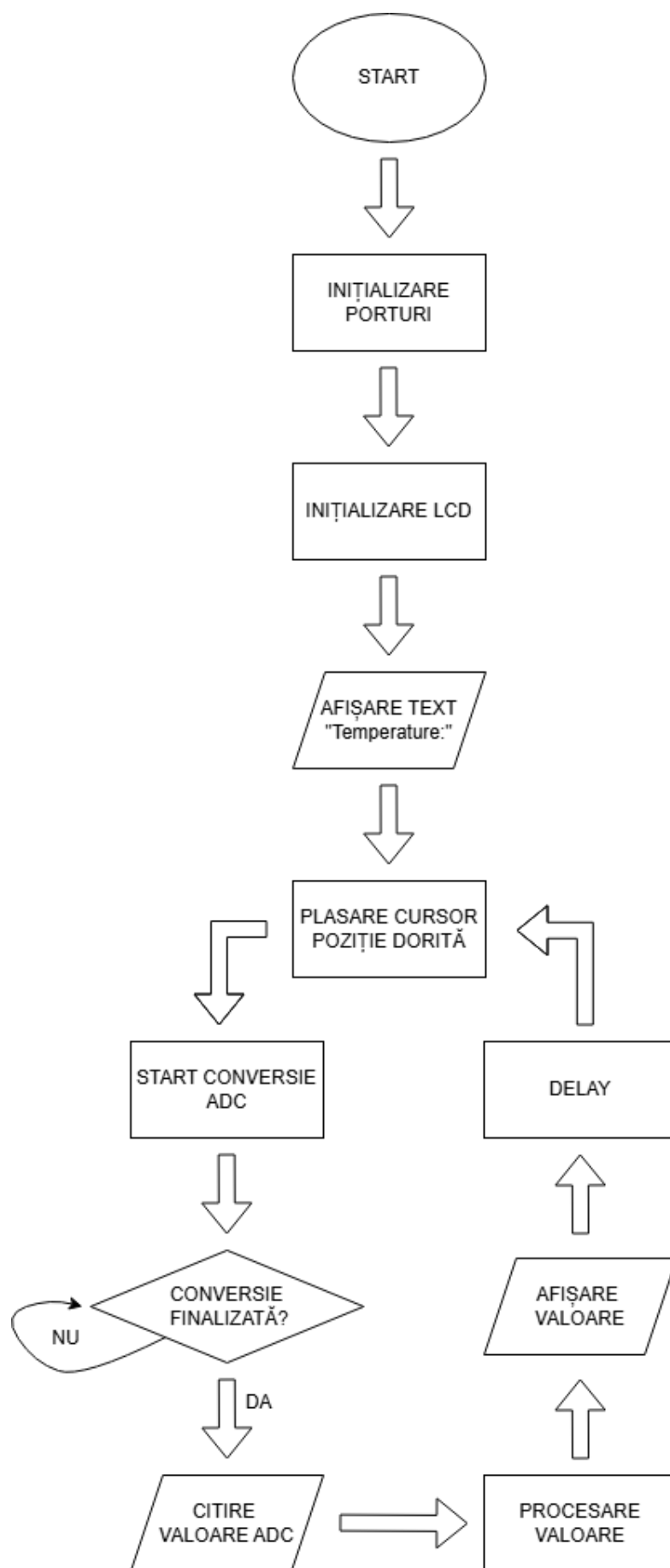


Figura 1: Diagrama bloc de funcționare

3. Preluarea Datelor de la ADC

Prima etapă fundamentală a sistemului o reprezintă achiziția datelor de la convertorul analog-digital. Pentru a asigura o citire corectă, microcontrolerul trebuie să respecte protocolul de comunicare care implică trei pași consecutivi: inițierea conversiei, așteptarea finalizării acesteia și preluarea datelor de pe magistrală.

Deoarece timpul de conversie al ADC0804 este mult mai mare decât timpul de execuție a unei instrucțiuni în microcontroler, sistemul implementează o tehnică de tip polling hardware. În loc să folosească o întârziere fixă, microcontrolerul monitorizează activ starea pinului de întrerupere.

Secvența instrucțiunilor este următoarea:

1. Se trimite un impuls LOW-HIGH pe pinul WR pentru a comanda începerea conversiei.
2. Se așteaptă într-o buclă cât timp pinul INTR este în starea HIGH.
3. Când INTR devine LOW, conversia este gata. Microcontrolerul transmite pe pinul RD un semnal LOW pentru a activa ieșirile ADC-ului, apoi citește valoarea brută (0 - 255) de pe Portul 1.

Funcția `read_adc()` implementează protocolul descris mai sus. În tabelul de mai jos sunt prezentate implementările funcției în limbajele C și Assembly.

<pre>unsigned char read_adc(){ unsigned char val; WR_ADC = 0; WR_ADC = 1; while(INTR == 1); RD_ADC = 0; val = ADC_DATA; RD_ADC = 1; return val; }</pre>	<pre>READ_ADC: CLR P3.6 NOP NOP SETB P3.6 WAIT: JB P3.2, WAIT CLR P3.7 NOP NOP MOV A, P1 SETB P3.7 RET</pre>
---	---

4. Interpretarea Datelor

Etapă de procesare are rolul de a transforma o valoare digitală abstractă într-o mărime fizică lizibilă. Această etapă necesită o atenție sporită la calcule, deoarece microcontrolerul este limitat la a lucra cu date de maxim 16 biți.

Valoarea brută preluată de la ADC este un număr întreg pe 8 biți, cuprins între 0 și 255. Sistemul este proiectat să măsoare temperaturi în intervalul 0°C - 40°C. Pentru a obține o precizie de o zecimală lucrând exclusiv cu numere întregi, valoarea finală este calculată direct în zecimi de grad, intervalul fiind 0 – 400.

Ecuatia inițială de mapare a intervalelor este:

$$T_{zecimi} = \frac{ADC * 400}{255}$$

Dar pentru a preveni depășirea capacității, adică overflow, la înmulțire și pentru a eficientiza împărțirea, fracția a fost simplificată matematic prin împărțirea la 5, rezultând ecuația care este implementată în cod:

$$T_{zecimi} = \frac{ADC * 80}{51}$$

În limbaj C, utilizarea tipului de date ‘unsigned int’, tip de date pe 16 biți, pentru rezultatul operațiilor previne automat erorile de tip overflow, iar extragerea cifrelor se realizează prin operatorii de modul ‘%’ și împărțire ‘/’.

În schimb, implementarea în limbaj Assembly expune o provocare tehnică: 8051 este un procesor pe 8 biți, astfel rezultatul înmulțirii, $ADC * 80$, va genera un rezultat pe 16 biți, stocat în doi regiștri separați: High Byte în registrul B și Low Byte în registrul A.

Împărțirea unui număr pe 16 biți la valoarea 51 necesită un algoritm de împărțire prin scăderi repetate, ținând cont de împrumutul (Carry) dintre octetul inferior și cel superior:

1. Se scade valoarea 51 din octetul inferior.
2. Se scade 0 împreună cu flag-ul de Carry din octetul superior.
3. Se contorizează de câte ori s-a putut realiza scăderea fără ca numărul pe 16 biți să devină negativ, rezultatul reprezentând temperatura în zecimi de grad

Odată obținut rezultatul, acesta este descompus:

- zecimile de grad (sutele matematice) se extrag prin scăderi repetate ale valorii 100;
- pentru unități și zecimale, restul rămas, fiind garantat mai mic decât 100, încapă într-un singur registru de 8 biți. Folosind instrucțiunea “DIV AB” pentru împărțirea la 10, se obține simultan câtul (unitățile) și restul (zecimale)

Tabelul de mai jos prezintă implementarea conversiei matematice și a divizării pe 16 biți în cele două limbaje:

temp = ((unsigned int)val * 80) / 51;	<pre> MOV B, #50h MUL AB MOV R0, B MOV R1, A MOV R2, #00h MOV R3, #00h DIV_51: CLR C MOV A, R1 SUBB A, #33h MOV R1, A MOV A, R0 SUBB A, #00h MOV R0, A JC DIV_DONE INC R3 MOV A, R3 JNZ DIV_51 INC R2 JMP DIV_51 DIV_DONE: MOV A, R2 MOV R0, A MOV A, R3 MOV R1, A </pre>
--	---

5. Afișarea Datelor pe LCD

Ultima etapă a sistemului constă în transmiterea datelor procesate către modulul LCD. Deoarece ecranul este un modul alfanumeric bazat pe standardul de codare ASCII, valorile brute obținute în etapa de procesare trebuie formate corespunzător înainte de a fi transmise pe magistrala de date.

Cifrele izolate anterior au valori numerice reale cuprinse între 0 și 9. Pentru ca driverul HD44780 al LCD-ului să afișeze corect aceste cifre, trebuie adunată valoarea hexazecimală 0x30, echivalentul caracterului '0'.

Afișarea temperaturii cu o precizie de o zecimală necesită inserarea caracterului punct, '.' cod ASCII 0x2E, între cifra unităților și a zecimalelor.

Pentru a îmbunătăți claritatea afișajului, s-a implementat o condiție astfel încât, dacă temperatura este mai mică de 10 grade, microcontrolerul transmite un spațiu, cod ASCII 0x20, în loc de cifra zero. Astfel sistemul afișează valoarea " 5.2" în loc de "05.2".

Tabelul de mai jos prezintă implementarea codului în cele două limbaje de programare.

<pre>if (tens > 0){ show_char(tens + '0'); } else{ show_char(' '); } show_char(ones + '0'); show_char('.'); show_char(tenth + '0');</pre>	<pre>PRINT_TEMP: MOV A, R4 JZ SPACE ADD A, #30h JMP PRINT SPACE: MOV A, #20h PRINT: ACALL SHOW_CHAR MOV A, R5 ADD A, #30h ACALL SHOW_CHAR MOV A, #2Eh ACALL SHOW_CHAR MOV A, R6 ADD A, #30h ACALL SHOW_CHAR</pre>
---	--

6. Simulări

Validarea funcționalității hardware și software a sistemului a fost realizată utilizând mediul de simulare Proteus. Acest mediu a permis simularea senzorului de temperatură cu valori precise și observarea comportamentului întregului lanț de achiziție, de la variația tensiunii analogice până la formatarea cifrelor de pe ecranul LCD.

Pentru a testa acuratețea afișajului, s-a efectuat o baleiere a temperaturii senzorului pe un interval restrâns, de la 16.0°C la 17.0°C cu un pas constant de 0.2°C. Scopul acestui test a fost verificarea liniarității răspunsului și identificarea eventualelor erori.

Rezultatele obținute în urma simulării sunt ilustrate în Figurile 2 – 7 și centralizate în tabelul următor:

Temperatură Senzor [°C]	Temperatură Afișată pe LCD	Eroare [°C]
16.0	16.0	0.0
16.2	16.1	- 0.1
16.4	16.4	0.0
16.6	16.6	0.0
16.8	16.7	- 0.1
17.0	17.0	0.0

Se observă o abatere de la valoarea temperaturii senzorului de $\pm 0.1^\circ\text{C}$. Această eroare este o limitare fizică strictă impusă de rezoluția convertorului analog-digital.

Sistemul utilizează un ADC0804, care are o arhitectură pe 8 biți. Astfel el poate genera doar 256 valori digitale distincte. Deoarece sistemul a fost proiectat să măsoare temperaturi de la 0°C la 40°C pe toată scala ADC-ului, rezoluția se calculează astfel:

$$Rezoluție_{ADC} = \frac{Temperatura_{MAX}}{Nr. Trepte} = \frac{40}{255} \approx 0.156^\circ\text{C}/pas$$

Această valoare demonstrează tehnic sursa erorii: microcontrolerul încearcă să afișeze temperatura cu un pas de 0.1°C, dar ADC-ul furnizează date cu un pas de 0.156°C.

S-au mai realizat două simulări pentru a testa valorile din extremitățile intervalului de temperatură. Rezultatele sunt ilustrate în Figurile 8 și 9.

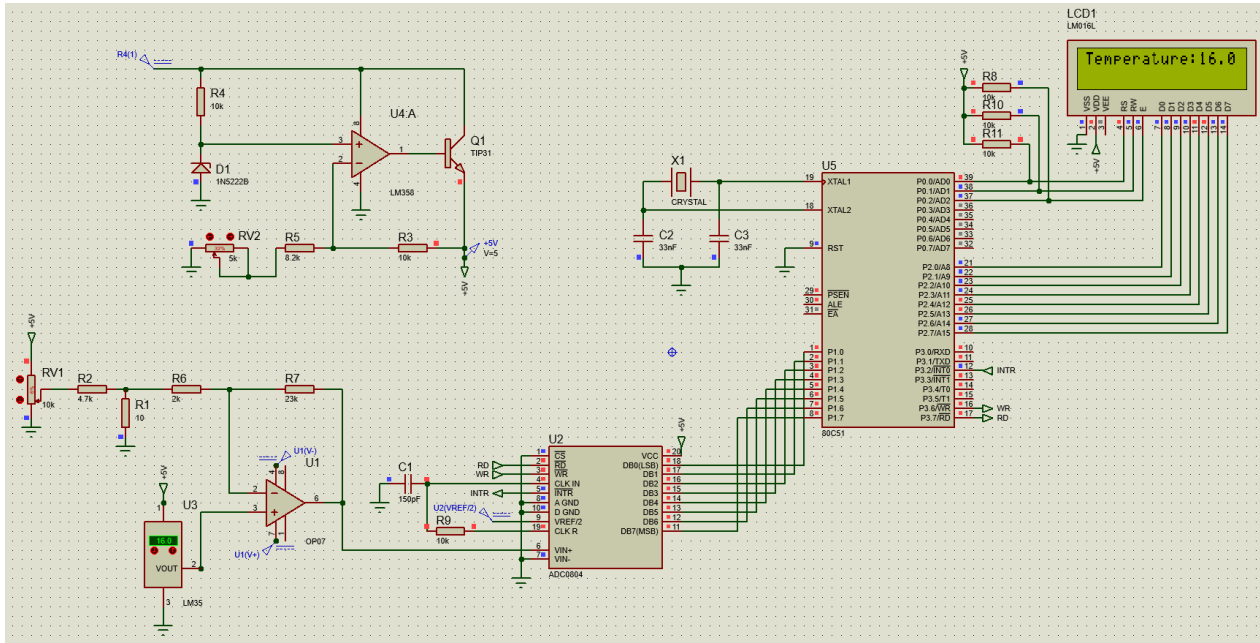


Figura 2: Simulare circuit – 16.0°C

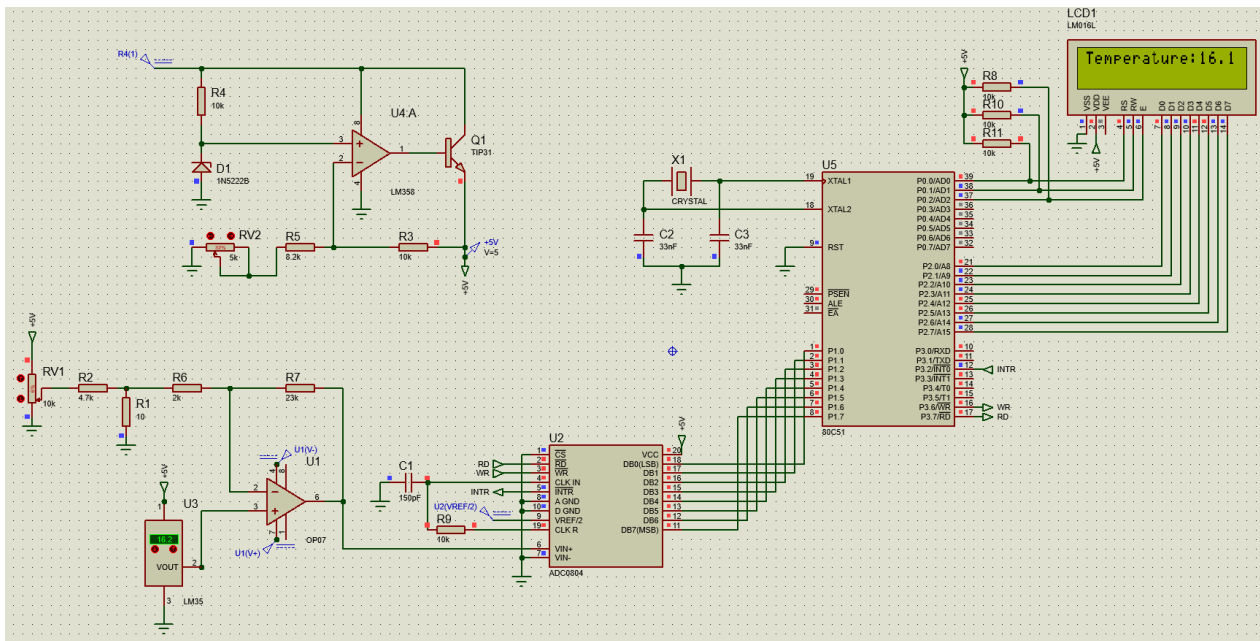


Figura 3: Simulare circuit – 16.2°C

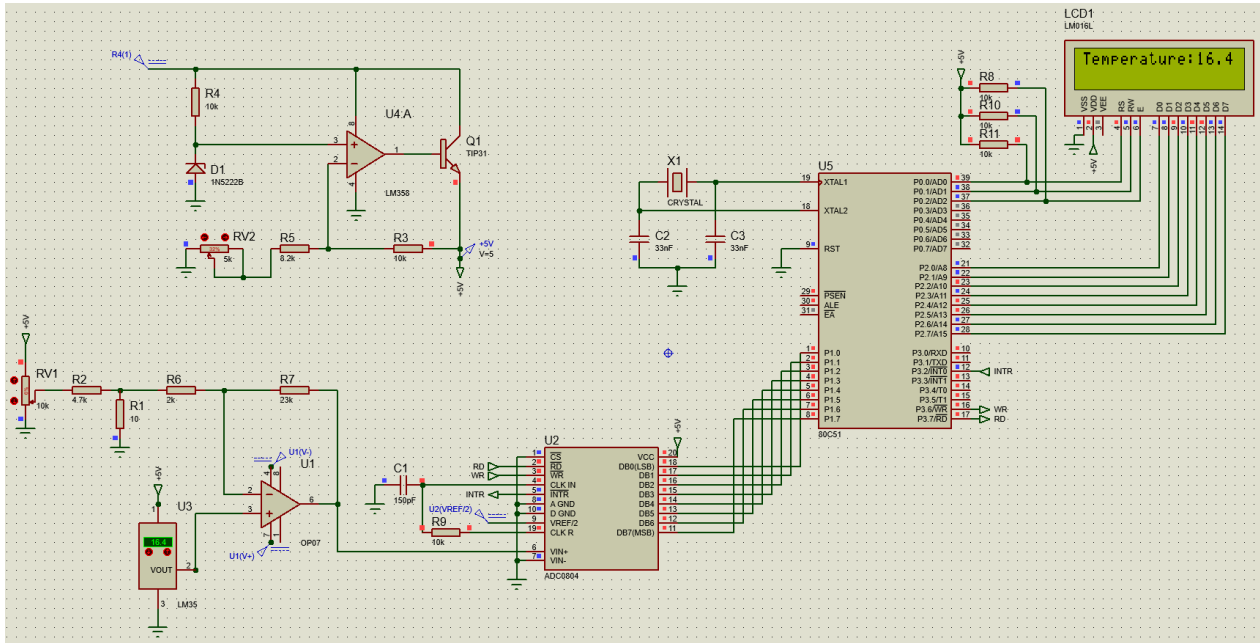


Figura 4: Simulare circuit – 16.4°C

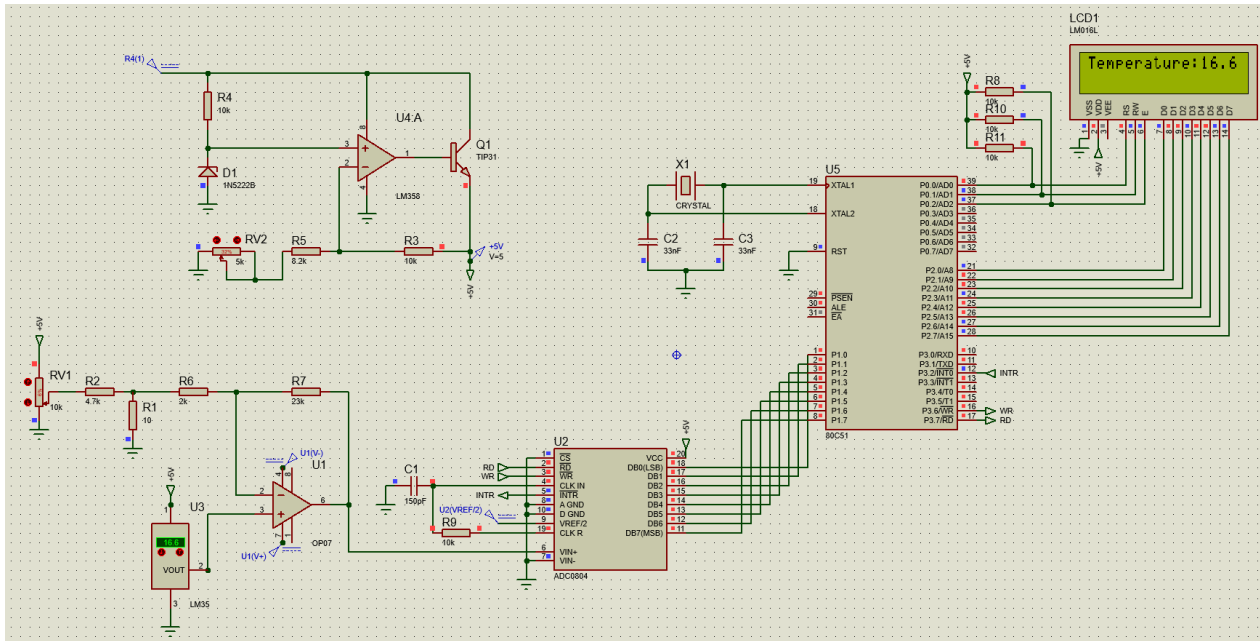


Figura 5: Simulare circuit – 16.6°C

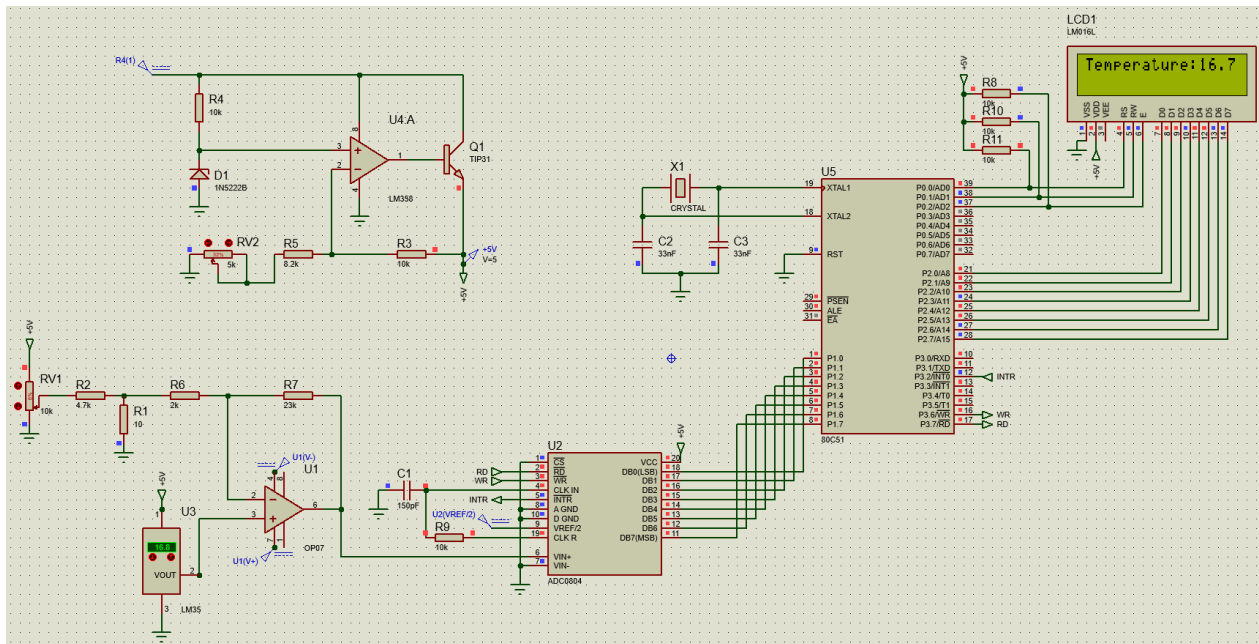


Figura 6: Simulare circuit – 16.8°C

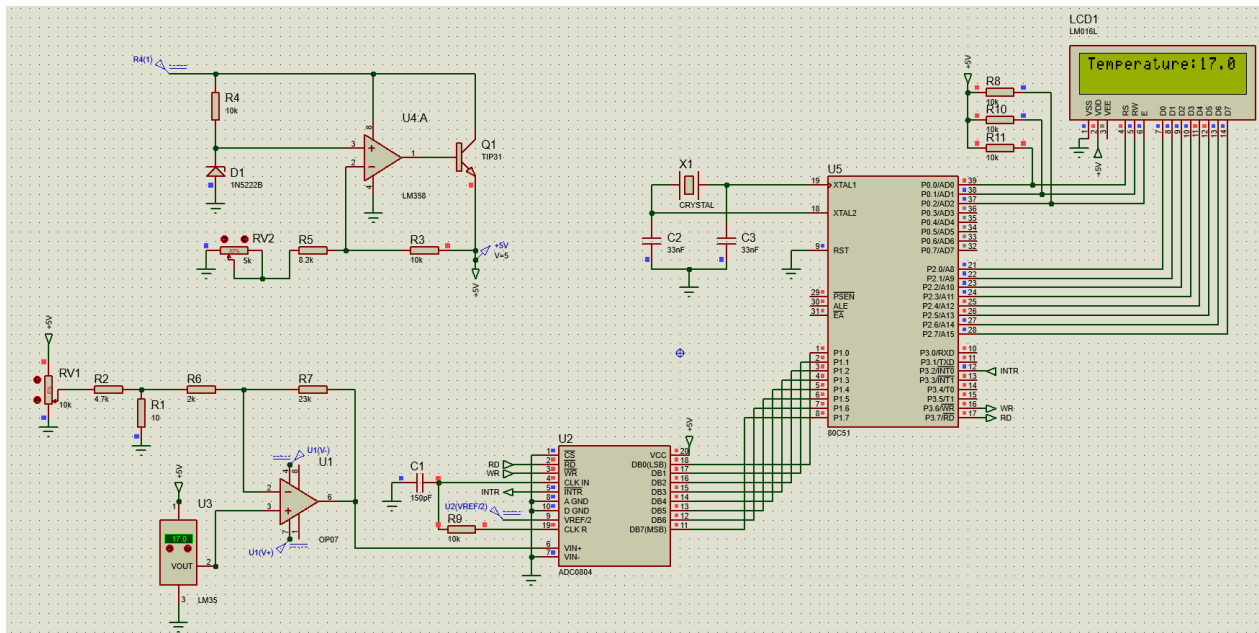


Figura 7: Simulare circuit – 17.0°C

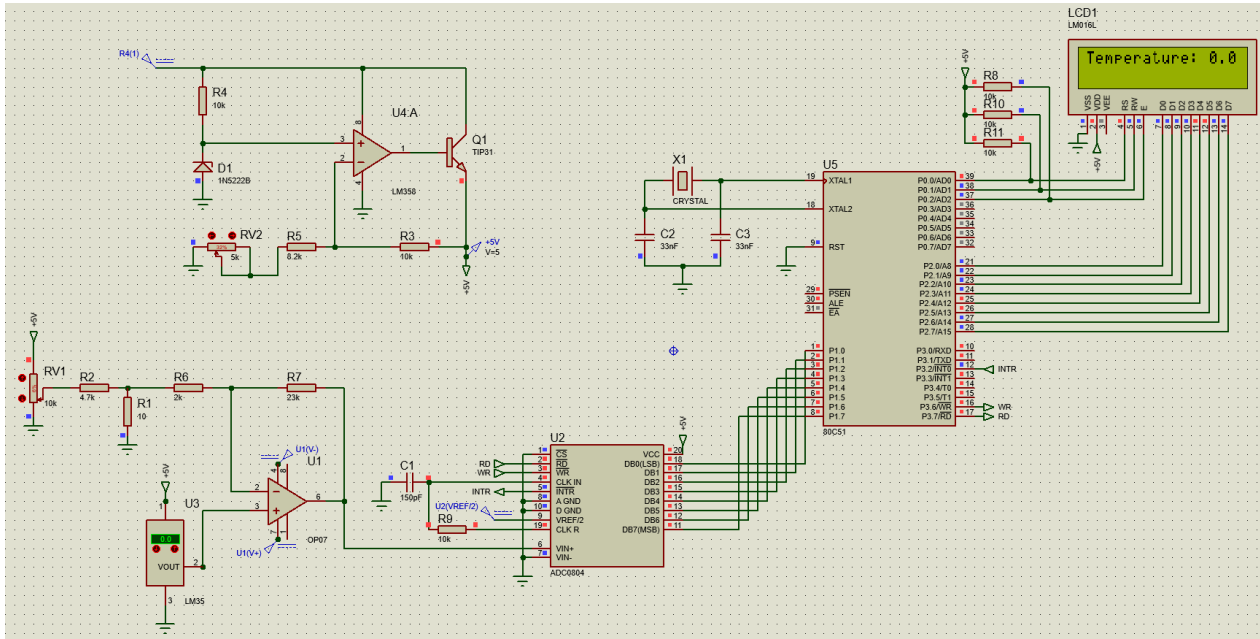


Figura 8: Simulare circuit - 0°C

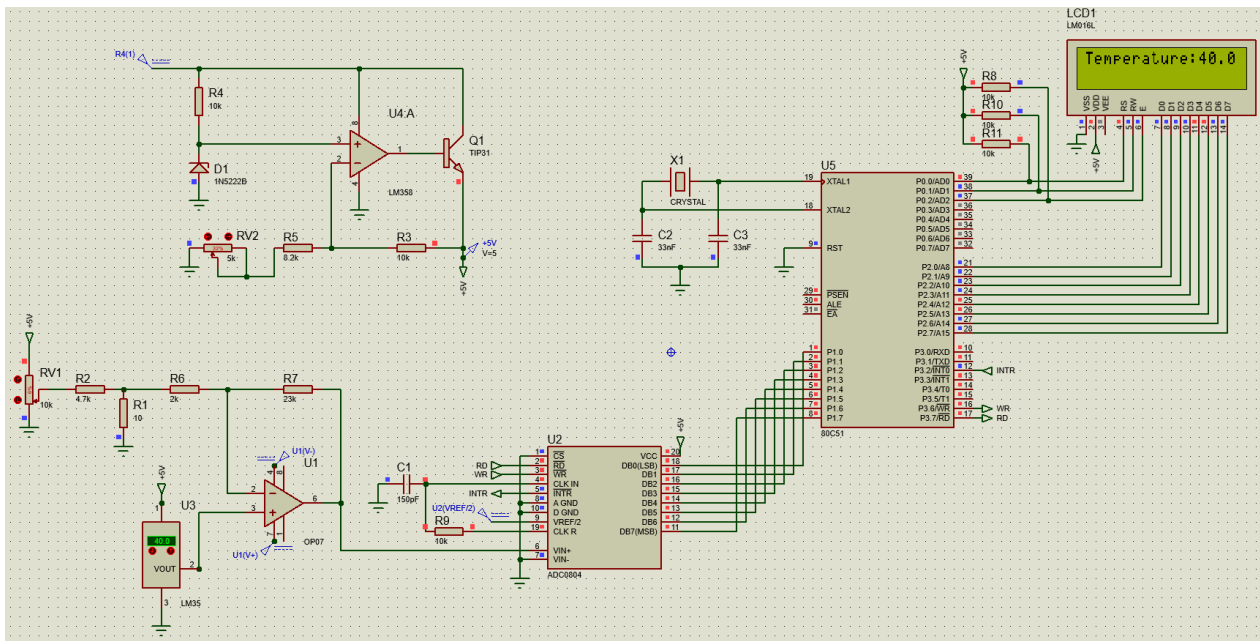


Figura 9: Simulare circuit - 40°C

7. Cod C & Assembly

```
#include <REG51.H>

#define ADC_DATA P1
#define LCD P2

sbit RS = P0^0;
sbit RW = P0^1;
sbit E = P0^2;
sbit WR_ADC = P3^6;
sbit RD_ADC = P3^7;
sbit INTR = P3^2;

unsigned char code string[] = "Temperature:";

void delay(int ms);
void command(int hex);
void setup_lcd();
void show_char(unsigned char x);
unsigned char read_adc();
void show_temp(unsigned char val);

void main(){
    ADC_DATA = 0x0FF;
    LCD = 0x00;
    RW = 0;
    INTR = 1;

    setup_lcd();

    while(1){
        command(0x8C);
        show_temp(read_adc());
        delay(300);
    }
}

void delay(int ms){
    unsigned int j, i;
    for(j = 0; j < ms; j++){
        for(i = 0; i < 112; i++){
        }
    }
}

void command(int hex){
    RS = 0;
    LCD = hex;
    E = 1;
```

```

    delay(1);
    E = 0;
}

void setup_lcd(){
    unsigned int i;

    command(0x38);
    delay(2);

    command(0x0E);
    delay(2);

    command(0x01);
    delay(2);

    command(0x06);
    delay(2);

    for( i=0; i < 0x0C; i++){
        show_char(string[i]);
    }
}

void show_char(unsigned char x){
    RS = 1;
    LCD = x;
    E = 1;
    delay(10);
    E = 0;
}

unsigned char read_adc(){
    unsigned char val;

    WR_ADC = 0;
    WR_ADC = 1;

    while(INTR == 1);

    RD_ADC = 0;
    val = ADC_DATA;
    RD_ADC = 1;

    return val;
}

void show_temp(unsigned char val){
    unsigned int temp;
    unsigned char tens, ones, tenth;

```



```
temp = ((unsigned int)val * 80 ) / 51;

tens = temp / 100;
ones = (temp % 100) / 10;
tenth = temp % 10;

if (tens > 0){
    show_char(tens + '0');
}
else{
    show_char(' ');
}

show_char(ones + '0');

show_char('.');
show_char(tenth + '0');
}
```

```

del EQU 99h

ORG 0000h
    JMP MAIN

ORG 0050h
    text_temp: DB "Temperature:",0

ORG 0100h

MAIN:
    MOV P1, #0FFh
    MOV P2, #00h
    CLR P0.1
    SETB P3.2

    ACALL SETUP_LCD

    MOV P2, #55h
    SETB P0.0

    LOOP:
        MOV A, #8Ch
        ACALL COMMAND
        ACALL READ_ADC
        ACALL SHOW_TEMP

        MOV A, #2Ch
        MOV B, #01h
        ACALL DELAY
    JMP LOOP

DELAY:
    PUSH 07
    PUSH ACC
    ORL A, B
    JNZ OK

    POP ACC
    POP 07
    RET
OK:
    POP ACC
DEC_LSB:
    ACALL ONEMIL
    DJNZ ACC, DEC_LSB

    CJNE A, B, DEC_MSB
    POP 07
    RET

```

```

DEC_MSB:
    DEC B

    SJMP DEC_LSB

ONEMIL:
    MOV R7, #del
    TIMER:
        NOP
        NOP
        NOP
        NOP
        DJNZ R7, TIMER
        NOP
RET

SETUP_LCD:

    MOV A, #38h
    ACALL COMMAND

    MOV A, #0Eh
    ACALL COMMAND

    MOV A, #01h
    ACALL COMMAND

    MOV A, #06h
    ACALL COMMAND

    MOV DPTR, #text_temp
    SHOW_TEXT:
        CLR A
        MOVC A, @A+DPTR
        JZ END_PRINT
        ACALL SHOW_CHAR
        INC DPTR
        JMP SHOW_TEXT
    END_PRINT:
RET

COMMAND:
    CLR P0.0
    MOV P2, A
    SETB P0.2
    ACALL ONEMIL
    ACALL ONEMIL
    CLR P0.2
RET

```

```

SHOW_CHAR:
    SETB P0.0
    MOV P2, A
    SETB P0.2
    MOV A, #0Ah
    MOV B, #00h
    ACALL DELAY
    CLR P0.2
RET

READ_ADC:
    CLR P3.6
    NOP
    NOP
    SETB P3.6
    NOP
    NOP
    NOP
    WAIT: JB P3.2, WAIT
    CLR P3.7
    NOP
    NOP
    MOV A, P1
    SETB P3.7
RET

SHOW_TEMP:
    MOV B, #50h
    MUL AB

    MOV R0, B
    MOV R1, A

    MOV R2, #00h
    MOV R3, #00h

    DIV_51:
        CLR C

        MOV A, R1
        SUBB A, #33h
        MOV R1, A

        MOV A, R0
        SUBB A, #00h
        MOV R0, A

        JC DIV_DONE
        INC R3
        MOV A, R3

```

```

    JNZ DIV_51
    INC R2
    JMP DIV_51
DIV_DONE:
    MOV A, R2
    MOV R0, A
    MOV A, R3
    MOV R1, A

    MOV R4, #00h
    MOV R5, #00h
    MOV R6, #00h

    LOOP_TENS:
        CLR C

        MOV A, R3
        SUBB A, #64h
        MOV R3, A

        MOV A, R2
        SUBB A, #00h
        MOV R2, A

        JC LOOP_TENS_DONE
        INC R4
        JMP LOOP_TENS
    LOOP_TENS_DONE:
        MOV A, R3
        ADD A, #64h
        MOV R3, A

    ONES_AND_TENTH:
        MOV A, R3
        MOV B, #0Ah
        DIV AB
        MOV R5, A
        MOV R6, B
    PRINT_TEMP:
        MOV A, R4
        JZ SPACE
        ADD A, #30h
        JMP PRINT
    SPACE:
        MOV A, #20h
    PRINT:
        ACALL SHOW_CHAR

        MOV A, R5
        ADD A, #30h

```

```
    ACALL SHOW_CHAR

    MOV A, #2Eh
    ACALL SHOW_CHAR

    MOV A, R6
    ADD A, #30h
    ACALL SHOW_CHAR
RET
END
```

8. Concluzii

În concluzie, obiectivul de a implementa software-ul unui sistem bazat pe microcontrolerul 8051 a fost îndeplinit cu succes. Sistemul este capabil să achiziționeze date de la senzorul de temperatură, să le proceseze și să le afișeze corect.

Dezvoltarea software-ului în două limbaje de programare distincte, C și Assembly, a scos în evidență un compromis clasic. Pe de o parte, limbajul C a oferit rapiditate în dezvoltare și o abstractizare matematică eficientă. Pe de altă parte, scrierea codului în Assembly a permis controlul absolut asupra timpilor de execuție și a regiștrilor.

Analiza erorilor din etapa de simulare a validat acuratețea algoritmilor software, demonstrând că singura abatere a sistemului, de $\pm 0.1^{\circ}\text{C}$, este datorată erorii de cuantizare a convertorului analog-digital pe 8 biți.

Precizia sistemului ar putea fi îmbunătățită prin înlocuirea ADC-ului cu o variantă pe 10 sau 12 biți.